Customer Churn Platform

Step-by-Step Learning Guide

This guide takes you from absolute scratch to understanding and rebuilding every piece of this production-grade machine learning platform.

Phase 1: Python Basics & Environment Setup

Before writing code, you need to understand how Python programs run and how they manage libraries.

- Core Goal: Understand how to run scripts and isolate dependencies.
- What to Study:

1. Virtual Environments (venv): Learn why we don't install packages globally. A virtual environment is a local sandbox folder containing a separate copy of Python and pip.

- Command to create: `python -m venv venv`
- Command to activate (Windows Powershell): `.\venv\Scripts\Activate.ps1`

2. Pip & Requirements: Learn how `pip install -r requirements.txt` downloads and maps packages inside your active virtual environment.

- Look at the codebase: Open and read `requirements.txt` to see the packages we are using.

Phase 2: Data Manipulation (Pandas & NumPy)

Pandas is the core library for working with tabular data (like spreadsheets or database tables) in Python.

- Core Goal: Learn to load, merge, and clean datasets.
- What to Study:

1. Pandas DataFrames: The primary 2D grid structure in Pandas.

2. Data Ingestion: How to load CSV files (`pd.read_csv`).

3. Merging DataFrames: How to combine datasets on a common column (`pd.merge(df1, df2, on='CustomerId')`).

4. Data Cleaning: How to locate and fill missing values (`df.fillna()`), clean spaces, and convert column data types (`pd.to_numeric()`).

- Look at the codebase: Open `src/data_ingestion.py` to see how we load the CSV and merge it with database queries.

Phase 3: Databases & SQL (SQLite)

SQL is the standard language used to communicate with relational databases.

- Core Goal: Learn how to write SQL queries and fetch records in Python.
- What to Study:

1. SQL SELECT Statements: How to request columns and filter rows (`SELECT * FROM services`).

2. SQLite3 in Python: Using Python's built-in `sqlite3` driver to open a connection and read tables.

3. Pandas Integration: Using `pd.read_sql_query(query, connection)` to load query results directly into a Pandas DataFrame.

- Look at the codebase: Review the SQL database connection code inside `src/data_ingestion.py`.

Phase 4: Machine Learning Pipelines (Scikit-Learn & XGBoost)

This is where raw data turns into mathematical models that can predict the future.

- Core Goal: Understand preprocessing, model training, and model evaluation.
- What to Study:

1. Features vs. Target: In customer churn, the target is what we want to predict (1 if the customer left, 0 if they stayed). The features are the details we use to predict it (tenure, contract type, charges).

2. Train-Test Split: Learn why we split data (e.g., 80% train, 20% test). We train models on the training set, and evaluate them on the test set to ensure the model can generalize to new customers it has never seen.

3. Preprocessing Pipelines:

- Scaling: ML models perform poorly when numerical features have wildly different ranges. We use `StandardScaler` to normalize them.
- Encoding: ML models only understand numbers, not words. We use `OneHotEncoder` to turn categorical values like Gender ("Male"/"Female") into columns of 0s and 1s.

4. Model Algorithms: Learn the intuition behind:

- Logistic Regression: A linear model that calculates probability boundaries (great baseline, fast, highly interpretable).
- Random Forest: An ensemble of Decision Trees that vote on predictions.
- XGBoost (Extreme Gradient Boosting): A highly optimized decision-tree boosting algorithm widely used in industry.

5. Model Evaluation Metrics:

- Accuracy: What percentage of predictions were correct?
- Precision: Out of all customers we predicted would churn, how many actually did?
- Recall: Out of all customers who actually churned, how many did we successfully flag?
- ROC-AUC: Area under the ROC curve. Measures how well the model separates the risk scores of churners vs. non-churners.
- Look at the codebase:
- Preprocessing details: `src/preprocessing.py`
- Model training logic: `src/train.py`
- Diagnostics and plots: `src/evaluate.py`

Phase 5: Building APIs (FastAPI)

FastAPI lets you expose your Python code to the internet as a web service.

- Core Goal: Create endpoints that accept JSON inputs, run prediction models, and return results.
- What to Study:

1. HTTP Methods: GET (fetching data/checking health) vs. POST (submitting data to get a prediction).

2. Pydantic: A library FastAPI uses to validate incoming JSON structure. It ensures the inputs match expected data types before running predictions.

3. Model Loading: Using `joblib.load()` to import a trained model inside the API process.

- Look at the codebase: Open `src/api.py` to inspect the Pydantic schemas and prediction endpoints.

Phase 6: Frontends & Dashboards (Streamlit)

Streamlit lets you build interactive web applications with pure Python code-no HTML, CSS, or JavaScript required.

- Core Goal: Build KPI cards, graphs, and form inputs that interact with your machine learning model.
- What to Study:

1. Layouts: Streamlit columns (`st.columns`), sidebars (`st.sidebar`), and forms (`st.form`).

2. Inputs: Sliders (`st.slider`), selection boxes (`st.selectbox`), and submit buttons.

3. Visualizations: Displaying Matplotlib/Seaborn figures (`st.pyplot`) and custom HTML.

4. Calling APIs: Using Python's `requests` library to send customer inputs to the FastAPI backend and fetch predictions in real time.

- Look at the codebase: See how user inputs are collected and predictions are displayed in `src/dashboard.py`.

Phase 7: Deployment & Containerization (Docker)

Docker packages your code, Python environment, and configuration into a single "container" that runs anywhere.

- Core Goal: Understand how to package and orchestrate multiple web services.
- What to Study:

1. Dockerfile: A recipe of steps to build an image (install OS utilities, copy code, install requirements, and expose ports).

2. Docker Compose: A configuration file (`docker-compose.yml`) used to coordinate multiple containers (FastAPI API and Streamlit Dashboard) to run together with shared folders (volumes).

- Look at the codebase:
- Image recipe: `Dockerfile`
- Coordination recipe: `docker-compose.yml`